

Metody Inteligencji Obliczeniowej

Sprawozdanie 7



Wydział Fizyki i Informatyki Stosowanej

Mikhail Shupliakou

16.05.2026

Spis treści

1	Wstęp	3
2	Zadanie 1	3
2.1	Proces wykonania zadania	3
2.2	Wyniki i wnioski	4
2.2.1	Analiza wyników liczbowych	4
2.2.2	Przebieg ewolucji	5
3	Zadanie 2	6
3.1	Proces wykonania zadania	6
3.2	Wyniki i wnioski	7
3.2.1	Analiza wyników	7
3.2.2	Wnioski	7

Algorytmy genetyczne

1 Wstęp

Algorytmy genetyczne (AG) to heurystyczne metody przeszukiwania przestrzeni rozwiązań, silnie inspirowane naturalnymi procesami ewolucji biologicznej oraz mechanizmami doboru naturalnego. Koncepcja ta, spopularyzowana w latach 70. XX wieku przez Johna Hollanda, opiera się na iteracyjnej ewolucji populacji tzw. osobników. Każdy z nich reprezentuje potencjalne rozwiązanie zadanego problemu, zazwyczaj zakodowane w postaci struktury przypominającej chromosom.

Proces ewolucyjny w algorytmach genetycznych realizowany jest poprzez cykliczne stosowanie trzech podstawowych operatorów:

- **Selekcji** – wyboru najlepiej przystosowanych osobników do reprodukcji na podstawie ich oceny,
- **Krzyżowania (reprodukcji)** – wymiany materiału genetycznego między wybranymi rodzicami w celu stworzenia nowego, potencjalnie lepszego potomstwa,
- **Mutacji** – wprowadzania drobnych, losowych zmian w genotypie, co pozwala na zachowanie różnorodności genetycznej w populacji i zapobiega przedwczesnemu utknięciu algorytmu w ekstremach lokalnych.

Kluczowym elementem sterującym działaniem algorytmu jest *funkcja przystosowania* (ang. *fitness function*), która pozwala na ilościową ocenę jakości każdego wygenerowanego rozwiązania. Dzięki swojej elastyczności i globalnemu charakterowi przeszukiwania, algorytmy genetyczne stanowią niezwykle skuteczne narzędzie w rozwiązywaniu złożonych problemów optymalizacyjnych – w szczególności tam, gdzie tradycyjne, deterministyczne metody matematyczne okazują się niewystarczające lub zbyt kosztowne obliczeniowo.

2 Zadanie 1

Celem pierwszego zadania jest implementacja algorytmu genetycznego, służącego do znalezienia globalnego maksimum zadanej funkcji matematycznej $f(x)$ w przedziale $[0, 1]$. Istotą ćwiczenia jest przeprowadzenie analizy porównawczej wpływu różnych mechanizmów i parametrów na skuteczność optymalizacji.

2.1 Proces wykonania zadania

Implementacja algorytmu genetycznego została zrealizowana w języku Python. W pierwszej kolejności zdefiniowano optymalizowaną funkcję celu, stanowiącą jednocześnie funkcję przystosowania (ang. *fitness function*):

```
import math

def f(x):
    return math.cos(80*x + 0.3) + 3*(x+1)**(-0.9) + 3*x**2 - math.sqrt(x)
```

Kluczowym elementem struktury programu jest klasa `Individual`, która reprezentuje pojedynczego osobnika w populacji. Zastosowano w niej 16-bitowy chromosom. Klasa ta posiada wbudowane mechanizmy dekodowania genotypu do wartości rzeczywistych z przedziału $[0, 1]$, obsługujące zarówno naturalny kod binarny, jak i kod Graya:

```
class Individual:
    L = 16 # dlugosc chromosomu
    def __init__(self, genes=None, use_gray=False):
        if genes is None:
            self.genes = [random.randint(0,1) for _ in range(self.L)]
        else:
            self.genes = genes[:]
        self.use_gray = use_gray
        self.fitness = None

    def decode(self):
        if self.use_gray:
            bin_genes = gray_decode(self.genes)
        else:
            bin_genes = self.genes
        val = int(''.join(str(b) for b in bin_genes), 2)
        return val / (2**self.L)
```

Proces ewolucji opiera się na standardowych operatorach genetycznych: jednopunktowym krzyżowaniu oraz mutacji bitowej. Do wyboru rodziców zaimplementowano dwie metody selekcji zgodnie z treścią zadania – selekcję ruletkową oraz selekcję progową (wybierającą spośród T najlepszych osobników).

Właściwy proces badawczy polega na iteracyjnym uruchamianiu algorytmu dla przygotowanej listy konfiguracji. Program testuje różne kombinacje kodowania, prawdopodobieństwa mutacji (p_m) oraz typu selekcji, wykonując po 10 niezależnych przebiegów dla każdego wariantu:

```
configs = [
    (False, 0.0, 'roulette', None),
    (False, 0.5, 'roulette', None),
    (True, 0.1, 'roulette', None),
    (False, 0.1, 'threshold', 5),
    # ... inne konfiguracje
]

for (gray, pm, sel, T) in configs:
    best_vals, hist = run_ga(use_gray=gray, pm=pm, selection_type=sel, T=T,
                             pop_size=50, generations=100, runs=10)
```

Taka struktura pozwoliła na automatyczne zebranie danych historycznych ze wszystkich iteracji w celu późniejszego wyznaczenia średnich wartości, odchyłeń standardowych oraz wygenerowania wykresów zbieżności algorytmu.

2.2 Wyniki i wnioski

2.2.1 Analiza wyników liczbowych

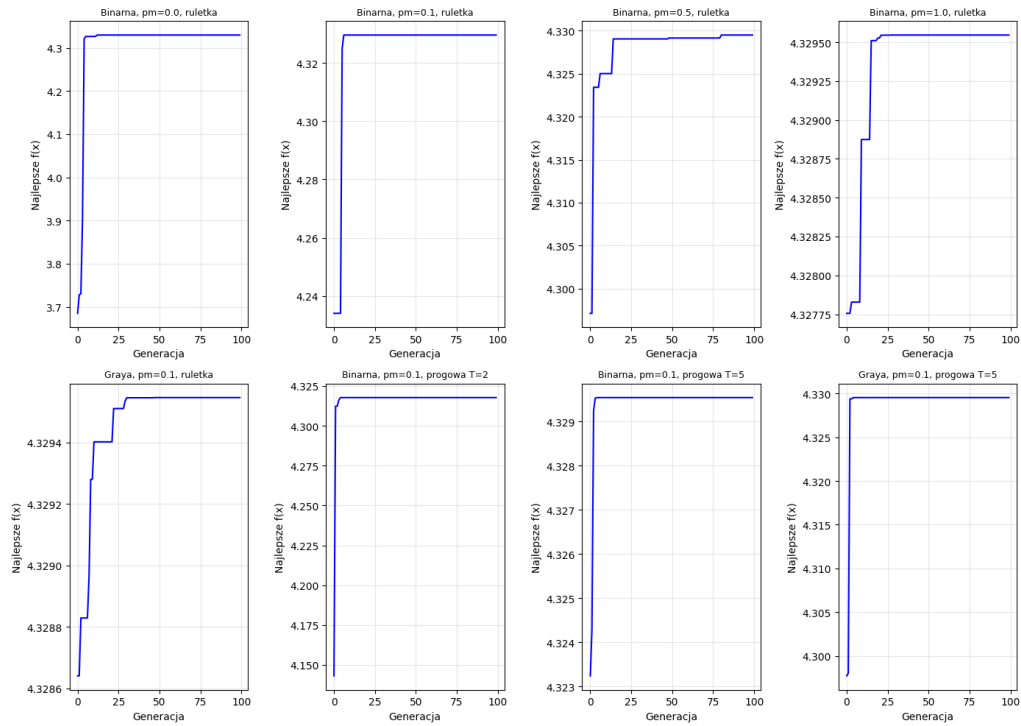
Na podstawie 10 niezależnych uruchomień algorytmu genetycznego (dla 100 generacji i populacji 50 osobników) zaobserwowano, że globalne maksimum badanej funkcji wynosi w przybliżeniu $f(x) \approx 4.3295$.

Zestawienie uzyskanych średnich i odchyłeń standardowych pozwala na wyciągnięcie następujących wniosków dotyczących wpływu poszczególnych parametrów:

- **Wpływ mutacji:** Mutacja jest kluczowa do odnalezienia globalnego optimum. W przypadku braku mutacji ($p_m = 0.0$) algorytm często utykał w maksimach lokalnych, co objawiło się niższym średnim wynikiem (~ 4.29) oraz widocznym odchyleniem standardowym (~ 0.09). Dodanie mutacji na poziomie $p_m = 0.1$ wystarczyło, aby w każdym przypadku stabilnie osiągać wartość optymalną (odchylenie standardowe równe 0). Zbyt wysoka mutacja ($p_m = 1.0$) przekształca algorytm w błędzenie losowe, jednak dzięki zastosowaniu mechanizmu elityzmu najlepsze rozwiązanie nie ulegało zniszczeniu.
- **Rodzaj selekcji:** Selekcja ruletkowa okazała się bardzo stabilna. W przypadku selekcji progowej zauważono zjawisko przedwczesnej zbieżności (ang. *premature convergence*) przy zbyt dużej presji selekcyjnej. Dla parametru $T = 2$ (wybór tylko spośród 2 najlepszych osobników) z kodowaniem binarnym algorytm ugrzązł w lokalnym optimum (wynik 4.2850). Złagodzenie presji do $T = 5$ pozwoliło na idealne zbieganie do globalnego maksimum.
- **Rodzaj kodowania:** Kodowanie Graya wykazało delikatną przewagę w sytuacjach ekstremalnych. Na przykład dla selekcji progowej $T = 2$, kodowanie Graya poradziło sobie ze znalezieniem optimum (4.3295), podczas gdy standardowe kodowanie binarne zawiodło. Wynika to z faktu, że w kodzie Graya sąsiednie wartości różnią się tylko jednym bitem, co ułatwia łagodne przeszukiwanie przestrzeni bez drastycznych skoków.

2.2.2 Przebieg ewolucji

Analiza wykresów zbieżności (Rysunek 1) potwierdza powyższe obserwacje. Dla optymalnych parametrów (np. $p_m = 0.1$, kodowanie Graya/Binarne, selekcja ruletkowa) algorytm wykazuje błyskawiczną zbieżność – znajduje rozwiązanie zbliżone do optymalnego często już w pierwszych 10–20 generacjach. W kolejnych iteracjach następuje jedynie drobne "dostrojenie" wyniku. Warianty bez mutacji wykazują na wykresach charakterystyczne spłaszczenia we wczesnych fazach ewolucji, co jest wizualnym dowodem na utknięcie populacji w lokalnych ekstremach.



Rysunek 1: Wykresy zbieżności algorytmu genetycznego dla wybranych konfiguracji. Na osi Y przedstawiono wartość najlepszego osobnika w danej generacji.

3 Zadanie 2

Cel zadania:

Zaprojektowanie i implementacja algorytmu genetycznego w celu znalezienia optymalnego przydziału budżetów ($B_1, B_2, B_3, B_4 \in [0, 400]$) dla czterech pizzerii, który zmaksymalizuje całkowity zysk sieci (funkcję Z).

3.1 Proces wykonania zadania

Implementacja algorytmu genetycznego dla problemu optymalizacji budżetów pizzerii została zrealizowana w języku Python. Ponieważ zmienne decyzyjne (B_1, B_2, B_3, B_4) mają charakter ciągły, zastosowano **kodowanie rzeczywistoliczbowe**. Genotyp każdego osobnika stanowi tablica czterech liczb zmiennoprzecinkowych, inicjalizowanych w dozwolonym przedziale $[0, 400]$.

Kluczowym elementem była implementacja funkcji przystosowania, symulującej zyski na siatce miasta. Do obliczania odległości wykorzystano metrykę taksówkową, a zyski sumowano z uwzględnieniem premii za bliską lokalizację:

```
def calculate_distance(x1, y1, x2, y2):
    return abs(x1 - x2) + abs(y1 - y2)

# Fragment logiki z funkcji fitness
if d < 2:
    z = 1.3 * (B / (0.5 * d + 4))
else:
    z = B / (0.5 * d + 4)
total_profit += z
```

Aby sprawnie przeszukiwać przestrzeń rozwiązań, dobrano operatory genetyczne odpowiednie dla wartości rzeczywistych. Za reprodukcję odpowiada **krzyżowanie arytmetyczne**, które tworzy potomków jako kombinację liniową genów rodziców. Wprowadzanie losowych zmian powierzono **mutacji gaussowskiej**, która modyfikuje geny poprzez dodanie szumu o rozkładzie normalnym. Zadbano również o mechanizm korygujący, który nie pozwala budżetom wykroczyć poza ustalone granice:

```
def mutate(individual, mutation_rate=0.1):
    for i in range(len(individual)):
        if random.random() < mutation_rate:
            individual[i] += random.gauss(0, 10) # Szum normalny
            # Utrzymanie wartosci w przedziale [0, 400]
            individual[i] = max(0.0, min(individual[i], 400.0))
    return individual
```

Do wyboru osobników predysponowanych do reprodukcji posłużyła **selekcja turniejowa**, gwarantująca odpowiednią presję selekcyjną. Cały proces ujęto w pętlę generacyjną z wbudowanym mechanizmem **elityzmu**, co zapewnia, że najlepsze znalezione rozwiązanie w danej epoce bezpowrotnie nie przepadnie wskutek mutacji czy krzyżowania:

```
# Elityzm: zachowanie najlepszego osobnika
best_idx = np.argmax(fitnesses)
new_population.append(population[best_idx])

# Tworzenie nowej generacji
while len(new_population) < pop_size:
    p1 = tournament_selection(population, fitnesses)
    p2 = tournament_selection(population, fitnesses)

    c1, c2 = crossover(p1, p2)
    new_population.append(mutate(c1))
    if len(new_population) < pop_size:
        new_population.append(mutate(c2))
```

3.2 Wyniki i wnioski

3.2.1 Analiza wyników

Uruchomienia algorytmu genetycznego (10 niezależnych iteracji) dla problemu optymalizacji budżetów pizzerii wykazały następujące wartości:

- Średnia wartość zysku (**Z**): 5859.06
- Odchylenie standardowe: 0.00

3.2.2 Wnioski

Powyższe metryki potwierdzają wysoką skuteczność opracowanej metody optymalizacji:

- **Absolutna stabilność i zbieżność:** Algorytm we wszystkich bez wyjątku próbach osiągnął identyczną maksymalną wartość funkcji celu ($Z \approx 5859.06$).

Zerowe odchylenie standardowe wskazuje na nienaganną powtarzalność i całkowitą odporność na utknięcie w ekstremach lokalnych.

- **Adekwatność operatorów genetycznych:** Kodowanie rzeczywistoliczbowe w połączeniu z mutacją gaussowską i krzyżowaniem arytmetycznym idealnie sprawdziło się w przeszukiwaniu ciągłej przestrzeni parametrów. Zapewniło to wymaganą precyzję podczas dostrajania wartości budżetów B_1, B_2, B_3, B_4 w dozwolonym przedziale $[0, 400]$.
- **Optymalny balans funkcji celu:** Algorytm z powodzeniem rozwiązał matematyczny konflikt między wzrostem zysków wynikającym z pokrycia sieci dostaw, a nieliniowym wzrostem kosztów rozbudowy $(B_1 + B_2 + B_3 + B_4)^{1.15}$. Zastosowanie selekcji turniejowej wraz z elityzmem zagwarantowało niezawodne zachowanie znalezionej równowagi przez wszystkie generacje, eliminując ryzyko utraty najlepszego osobnika.

Zaproponowana architektura algorytmu genetycznego w pełni radzi sobie z postawionym zadaniem, wykazując maksymalną niezawodność podczas poszukiwania globalnego optimum ciągłej funkcji zysku.